README Sprint S04 · Python para IA

Pregunta guía: ¿cómo automatizamos lo que hemos hecho a mano?
Fechas: 3–4 de noviembre · **Horas:** 9 h (2 h teoría · 4 h práctica · 3 h PBL)
📄 **Este README en PDF** (para imprimir; descárgalo para que funcionen los enlaces): a color · ecofriendly

> **Sigue el orden de arriba abajo y abre solo lo que pone en cada fila.** Los demás documentos del sprint son para consultar si te atascas (apartado «Si te atascas»). Si el tiempo aprieta, haz solo los ejercicios marcados «en clase»: el resto es para casa.

Qué aprenderás

1. Escribir **Python** básico en el navegador y en Colab (variables, decisiones, bucles) y crear **funciones y módulos** con pruebas.
2. Representar datos con **listas, diccionarios y conjuntos**, y leer y escribir **CSV y JSON** controlando los errores.
3. Entregar una herramienta con **pruebas y Git**: es lo que haréis en el reto «Herramientas internas».

Día 1 · Martes 3/11

☐	Qué haces	Abre	Min
☐	🔔 Ceremonia · Planning del reto «Herramientas internas» y test de nivel de partida (5 preguntas, anónimo)	—	10
☐	📖 Teoría · Python: entorno (venv, pip, Jupyter, Git) y lenguaje	Presentación S04.1	25
☐	💻 Práctica · Curso Python desde cero ★ (días 2–3): type(), indexar y formatear	Abrir	40
☐	💻 Práctica · Curso Python desde cero ★ (día 4): listas, tuplas, dicts e if/else	Abrir	50
☐	💻 Práctica · Curso Python desde cero ★ (día 5): for, range() y funciones	Abrir	45
☐	💻 Práctica · Curso Python desde cero ★ (ampliación): try/except, CSV y JSON	Abrir	30

En clase: 3 h 20 min.

Al acabar el día, deberías poder decir:

☐ Mi entorno funciona y sé ejecutar código en el navegador y en un cuaderno.
☐ Sé escribir una función corta y leer un CSV.

Día 2 · Miércoles 4/11

☐	Qué haces	Abre	Min
☐	🎯 Reto · Preparar la carpeta y ver que las pruebas fallan → rellenar las funciones una a una → exportar y ejecutar → probar que no se rompe → README y Git (pasos 1–5, versión mínima; el docente modela `leer_reservas` en directo los primeros 15 min)	Reto en una página	100
☐	🔔 Ceremonia · Entrega en `sprint-04/` y autoevaluación y coevaluación del Sprint 4 (la review y la retro se hacen junto con las del Sprint 5)	—	15

En clase: 1 h 55 min.

El reto en pocas palabras

Qué os pide el cliente. El equipo de reservas de TurisData calcula sus cifras semanales **a mano en una hoja de cálculo**. Es lento y cada persona lo hace un poco distinto. Os pide una **herramienta interna**: un pequeño programa que **lea el fichero de reservas, calcule tres cifras importantes (KPIs), avise con claridad si el fichero está mal y guarde los resultados** en un formato que otros sistemas puedan leer.

Qué vais a entregar: un repositorio Git con `kpis.py`, `test_kpis.py`, `README.md` y un `salida.json` generado con el CSV real.

Núcleo y extra. Núcleo: las funciones obligatorias hasta que `test_kpis.py` pase y el README. Extra: las partes opcionales (POO y Git básico).

Para trabajar: reto en una página. Para consultar: ficha completa del reto (pasos, rúbrica, ayudas y errores frecuentes). Plantillas, datos y autoevaluación: `3_Reto/4_Plantillas_y_entrega/`.

✅ Antes de cerrar el sprint (Definition of Done)

Archivos subidos en el repositorio del equipo, carpeta `sprint-04/` (marca cada uno cuando lo veas en GitHub): ☐ `kpis.py` · vuestro código, con las funciones completas
☐ `test_kpis.py` · el dado, **sin cambios**
☐ `README.md` · rellenado, con la sección «Revisión con la rúbrica»
☐ `salida.json` · generado con el CSV real
☐ `autoevaluacion_nombre.md` · **uno por persona** (la coevaluación va dentro del mismo fichero): plantilla

Total en `sprint-04/`: 4 ficheros del equipo + 1 hoja por persona (un equipo de 3 sube 7). Aparte: el CSV en `data/` si aún no está, y los extras (`kpis_pandas.py`, pruebas propias) solo si los hacéis. La *review* y la *retro* no se suben.

Comprobaciones antes de dar el sprint por cerrado: ☐ `python test_kpis.py` pasa todas las pruebas
☐ `python kpis.py reservas_turisdata.csv salida.json` genera el JSON con los 3 KPIs y `calidad`
☐ Funciones con docstring y README con las definiciones de los KPIs
☐ Al menos 3 *commits* en el repositorio, de personas distintas
☐ Alguien que **no escribió esa parte** ha revisado el código y el README con la rúbrica y lo ha anotado en el README

En casa (cuando puedas)

- Los ejercicios de cada cuaderno que **no** se hacen en clase. Se autocorrigen, y debajo llevan una «Solución de referencia» plegada por si te quedas atascado.
- Curso «Python desde cero»: lo que os falte de la **ruta mínima** ★ y los proyectos «Agenda telefónica» y «Tablas de multiplicar» (abrir el curso). Al acabar cada día, **exportad vuestro progreso** (botón «Exportar mi progreso») y enviad una captura del contador.
- Refuerzo en Colab: los cuadernos `S04_01`, `S04_02`, `S04_03` y `S04_04`, empezando por los ejercicios «en clase».
- Cuaderno completo en casa: `S04_05_poo_buenas_practicas`
- Si quieres repasar la teoría con calma: apuntes del sprint.

Si te atascas

1. Lee primero la parte «Si os atascáis» de la ficha completa del reto.
2. Mira el código explicado del cuaderno en el que estás.
3. Pregunta a tu equipo y, después, al docente. Si usas IA, cuenta en el README qué te dijo y qué comprobaste.

Todo el material, por carpetas

Las carpetas están **numeradas en el orden en que las usas**. Cada documento está en `.md`; sus PDF (color y ecofriendly) están en la subcarpeta `pdf/`.

Carpeta	Qué hay
`1_Teoria/`	Las presentaciones del sprint y los apuntes para repasar
`2_Practica/`	Los cuadernos (Colab) y su explicación línea a línea en `codigo_explicado/`
`3_Reto/`	`1_Reto_en_una_pagina` · `2_Ficha_completa` · `3_Laminas` · `4_Plantillas_y_entrega`
`4_Referencia/`	Infografías del sprint

Cómo se trabaja un reto, en todos los sprints: método de trabajo.

Temario oficial de este sprint

Unidad	Conceptos que entran
UD 1b.1 · Puesta en marcha del entorno	Instalación de Python, entornos virtuales, pip, Jupyter, VS Code; Git básico
UD 1b.2 · Fundamentos del lenguaje	Sintaxis, tipos, operadores, input/print, condicionales, bucles
UD 1b.3 · Estructuras de datos y funciones	Listas, tuplas, diccionarios, conjuntos, comprensiones · Funciones, parámetros, lambda, args/kwargs
UD 1b.4 · Módulos, ficheros, excepciones y POO	Importación de módulos, lectura y escritura de ficheros (CSV, JSON) · Excepciones; clases y objetos (comparativa con Java)

Y ahora: Sprint 5 · Matemáticas y estadística en código (miércoles 4/11).

9 h (2 h teoría · 4 h práctica · 3 h PBL).